USN No: 23BTRCL014

Name: DHARUN P

Experiment 5: Unified Stream Processing with Apache Flink

Topic

Stream Processing with Apache Flink using the PyFlink DataStream API: local installation, core concepts (sources, transformations, keyed streams, windows), and a live word-frequency demonstration over bounded, unbounded, and windowed streams — all running natively on Windows without a separate cluster.

Objective

To install and configure Apache Flink through its Python API (PyFlink) on a native Windows machine, to understand Flink's unified processing model in which a batch is treated as a special case of a stream, to learn the core DataStream abstractions — sources, transformations (flat_map, key_by, reduce), and windows — and to build and run a live word-frequency application over a bounded collection, an unbounded continuous stream, and a time-windowed stream, so as to distinguish Flink's true event-at-a-time stream processing from the messaging of Experiment 1 (Kafka) and to compare it with the topology model of Experiment 2 (Storm).

Introduction to Apache Flink

Apache Flink is an open-source, distributed engine for stateful computations over data streams. Where Experiment 1 (Kafka) showed records being moved reliably from producers to consumers, and Experiment 2 (Storm) showed those records being processed continuously by a topology of spouts and bolts, Flink provides the same kind of continuous, stateful processing but through a single unified model and high-level APIs that require far less boilerplate than Storm.

The central idea of Flink is unification: a bounded dataset (a fixed file or list) is simply a stream that happens to end, while an unbounded dataset (a socket, a Kafka topic, a monitored folder) is a stream that never ends. The exact same operators apply to both. This mirrors the mental model contrast with Apache Spark: in Spark one thinks of a stream as many small batches, whereas in Flink one thinks of a batch as just a bounded stream. Flink processes each record the instant it arrives (true event-at-a-time processing), rather than in micro-batches.

A Flink program written with the DataStream API has three parts. A source produces the data — for example from_collection for a fixed list, or a FileSource that continuously monitors a directory for an unbounded feed.

Transformations reshape the stream: flat_map splits each line into words, key_by groups records by a key (the analogue of Spark's reduceByKey key), and reduce combines records sharing that key. Finally a sink consumes the results, here simply printing them. Nothing executes until env.execute() is called — the pipeline is lazy, exactly like a Spark action triggering a job.

Unlike Storm, which requires separate Nimbus, Supervisor and UI daemons to be started before a topology can be submitted, PyFlink launches its own local mini-cluster inside the Python process when env.execute() runs. This means a complete Flink job can be developed and executed on native Windows with nothing more than Java and the PyFlink package — no cluster scripts, no external services. This experiment builds on the streaming concepts of Experiments 1 and 2 and directly supports CO1 and CO2 of the course.

SET 1: Prerequisites and Verifying the Environment

Goal: *Confirm that a supported Java version and a PyFlink-compatible Python version are available, taking care to select the correct Python when several are installed.*

Task 1 — Verify Java

Apache Flink 2.x requires Java 11 or later (Java 17 is recommended). Check the installed version:

```
java -version
```

Sample Output:

```
openjdk version "17.0.20" 2026-07-21 OpenJDK
Runtime Environment Temurin-17.0.20+8 ...
OpenJDK 64-Bit Server VM Temurin-17.0.20+8 ...
```

Task 2 — Verify the Python version (important on multi-version systems)

PyFlink ships prebuilt wheels only for Python 3.9, 3.10, 3.11 and 3.12 — it does NOT yet support 3.13. On a machine with several Python versions installed, the default python may be an unsupported one, in which case pip will try to build PyFlink from source and fail. Use the Windows py launcher to list the installed versions and pick a supported one explicitly.

```
py -0
```

*Sample Output (the * marks the default — here the unsupported 3.13):*

```
 -V:3.13 *        Python 3.13 (64-bit)
 -V:3.11          Python 3.11 (64-bit)
 -V:3.10          Python 3.10 (64-bit)
 -V:3.7           Python 3.7 (64-bit)
```

```
Known Windows issue — read before proceeding

If the default Python (marked * above) is 3.13 or newer, a plain
"pip install apache-flink" fails with a build/subprocess error,
because no 3.13 wheel exists and pip falls back to compiling from
source. Fix: always create the virtual environment with a supported
version explicitly, e.g. py -3.11, as shown in the next SET.
```

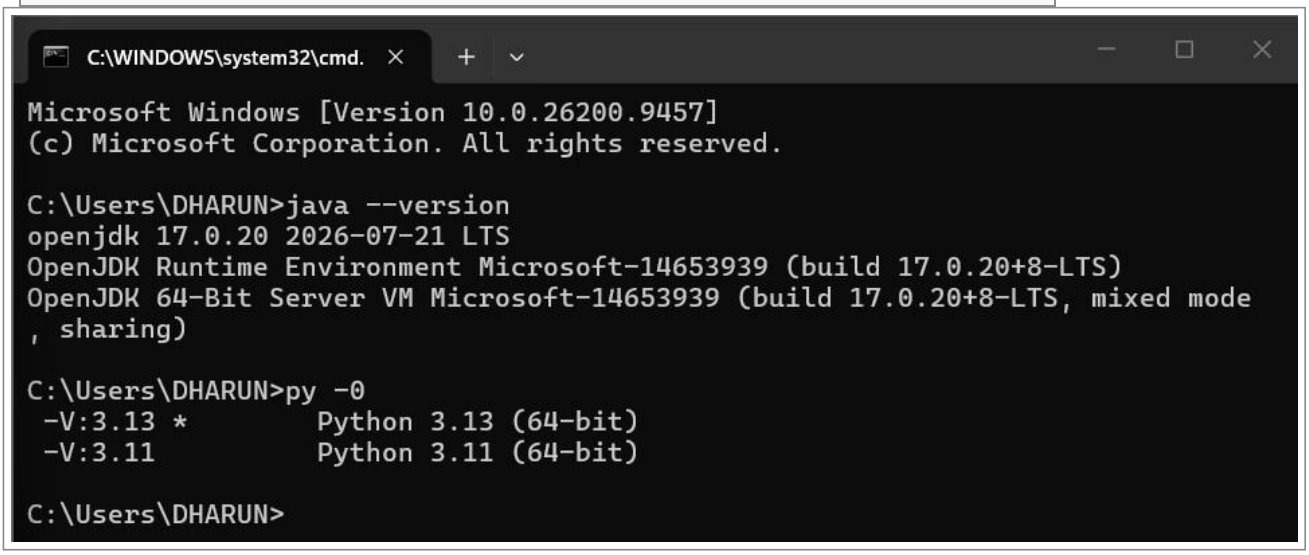


Figure 1: Verifying the Java version (OpenJDK 17.0.20) and listing installed Python versions with py -0

Result: Java 17 and a PyFlink-supported Python (3.11) are both present, and the correct Python interpreter has been identified for use in the virtual environment.

SET 2: Installing and Verifying PyFlink

Goal: *Create an isolated virtual environment pinned to Python 3.11, install PyFlink into it, and confirm the installation.*

Task 1 — Create and activate a virtual environment with Python 3.11

A virtual environment keeps PyFlink and its dependencies separate from the system Python. Create it with the supported interpreter, then activate it. The prompt will show (flink-env) once active.

```
py -3.11 -m venv flink-env
flink-env\Scripts\activate
python --version
```

Sample Output:

```
Python 3.11.9 (the version MUST read 3.11.x — this confirms
the venv uses the  correct interpreter before installing)
```

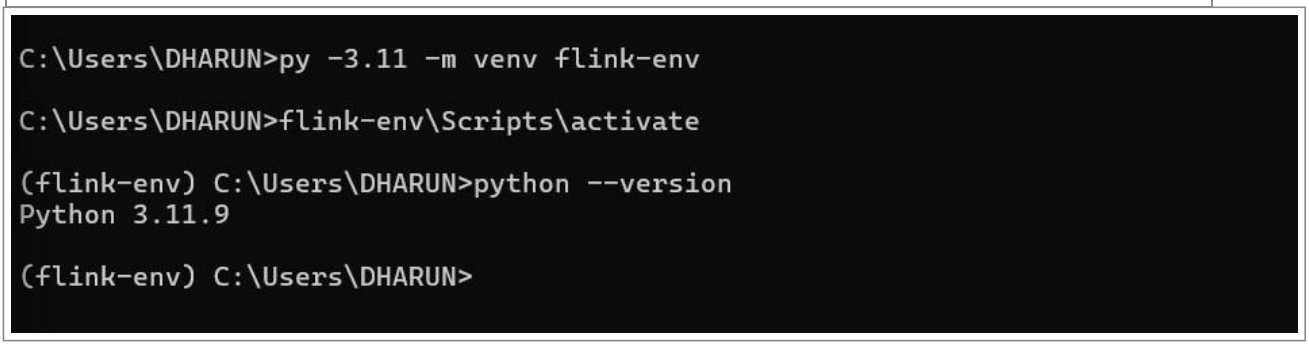


Figure 2: Creating and activating the flink-env virtual environment (Python 3.11.9)

Task 2 — Install PyFlink

With the environment active, upgrade pip and install the apache-flink package (about 300 MB, as it bundles the Flink runtime). Because the venv uses Python 3.11, pip now downloads the prebuilt wheel instead of building from source.

```
python -m pip install --upgrade pip
pip install apache-flink
```

Sample Output:

```
Successfully installed apache-flink-2.3.0 apache-flink-libraries-2.3.0 ...
```

Task 3 — Verify the installation

```
python -c "import pyflink; print(pyflink.__version__)"
```

Sample Output:

```
2.3.0
```

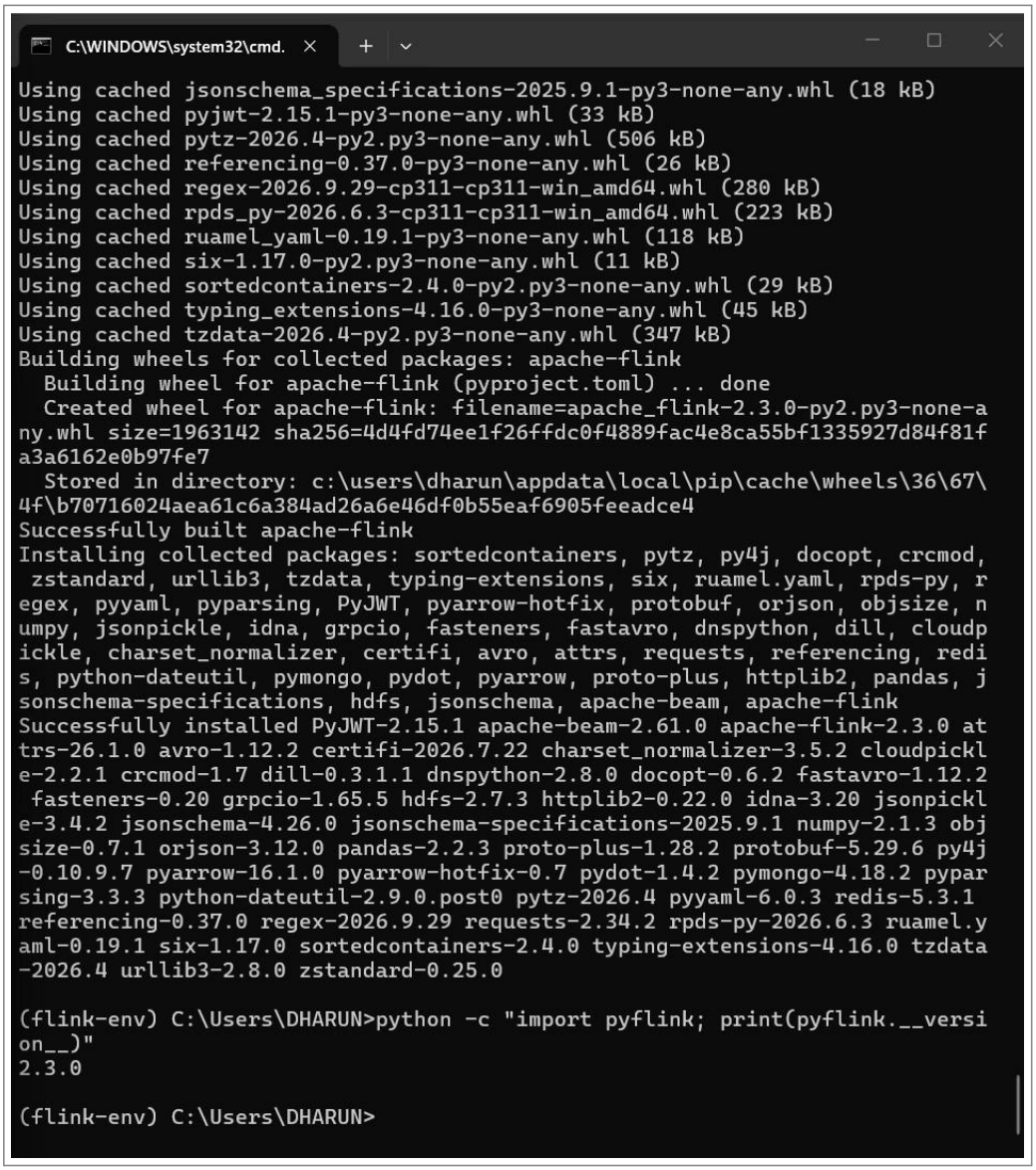


Figure 3: Installing apache-flink 2.3.0 and verifying the PyFlink version

Result: PyFlink 2.3.0 is installed inside an isolated Python 3.11 environment and imports successfully, confirming that Flink's Python API and its bundled runtime are ready — with no separate cluster to start.

SET 3: Bounded Stream — Word Count from a Collection

Goal: *Build and run a first Flink job over a fixed in-memory list (a bounded stream) to learn the DataStream operators and the lazy execution model.*

Task 1 — Create the project folder

```
mkdir C:\Flink
cd C:\Flink
```

Task 2 — Create wordcount_collection.py

This job creates a bounded stream from a list, splits each line into (word, 1) tuples with flat_map, groups by word with key_by, and sums with reduce.

```
from pyflink.common import Types from pyflink.datastream import
StreamExecutionEnvironment  # Entry point — analogous to SparkSession
env = StreamExecutionEnvironment.get_execution_environment()
env.set_parallelism(1)   # single output stream, keeps results
readable  # A bounded stream from an in-memory list data =
env.from_collection(     ["apache flink is fun",
     "flink flink flink",
"kafka storm spark flink"],
type_info=Types.STRING() )
 # Split each line into (word, 1)
tuples def tokenize(line):     for
word in line.split():
        yield (word, 1)  counts = (data     .flat_map(tokenize,
output_type=Types.TUPLE([Types.STRING(), Types.INT()]))
.key_by(lambda x: x[0])                # group by word
    .reduce(lambda a, b: (a[0], a[1] + b[1]))  counts.print()
env.execute("word_count_collection")      # nothing runs until
here
```

Task 3 — Run the job

```
cd C:\Flink
python wordcount_collection.py
```

Sample Output (some JVM startup log lines are normal and omitted here):

```
(apache,1)
(flink,1)
(is,1)
(fun,1)
(flink,2)
(flink,3)
(flink,4)
(kafka,1)
(storm,1)
(spark,1)
(flink,5)
```

Notice that flink is emitted at every intermediate count (1, 2, 3, 4, 5) rather than only as a final total. This is Flink's event-at-a-time model: reduce produces an updated result on each record, not a single aggregate at the end. key_by + reduce here is the analogue of Spark's reduceByKey.

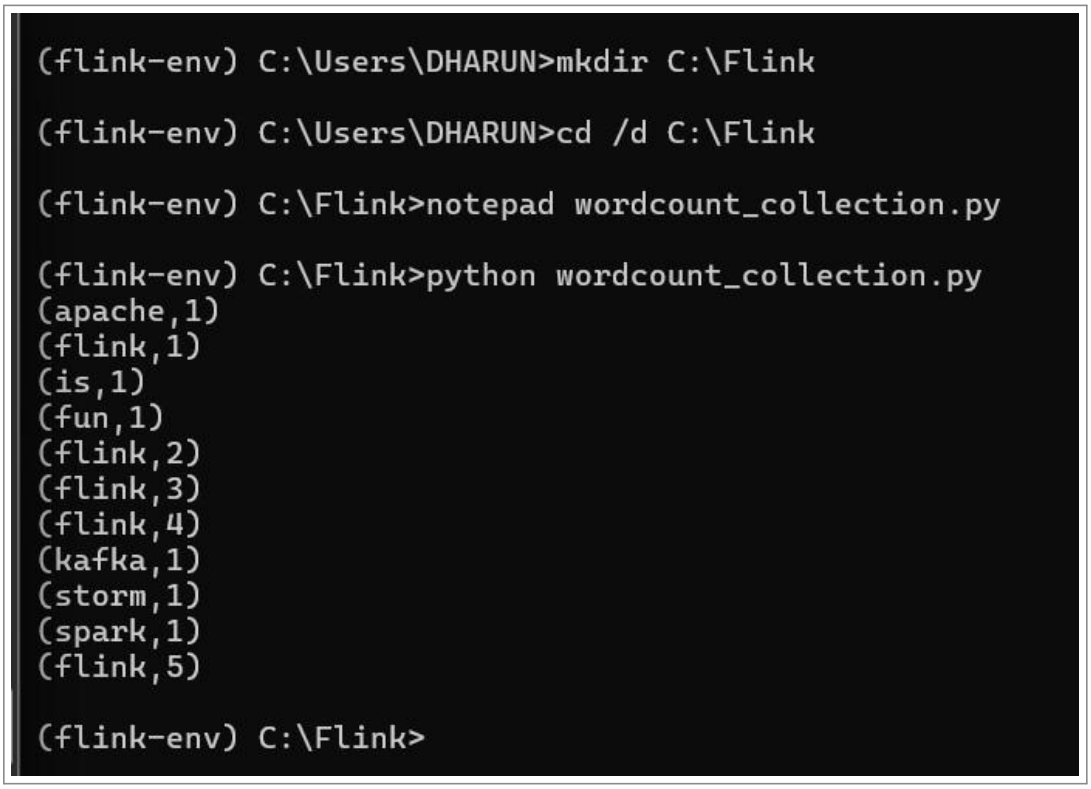


Figure 4: Running wordcount_collection.py — bounded stream word count output

Result: A complete Flink job ran on the embedded mini-cluster and produced a correct word count, with flink reaching 5. The running intermediate counts demonstrate true per-record stream processing, and the job finished on its own because the collection source is bounded.

SET 4: Unbounded Stream — Continuous Streaming Word Count

Goal: *Replace the fixed list with a source that never ends — a directory Flink continuously monitors — so the same operators now process an unbounded, live stream. New text files dropped into the folder are counted in real time.*

Note: Flink 2.x removed the old socket_text_stream source, so the modern FileSource with continuous monitoring is used instead. It needs no external service and works entirely on native Windows.

Task 1 — Create the input folder to be monitored

```
cd C:\Flink mkdir
stream_input
```

Task 2 — Create wordcount_stream.py

```
from pyflink.common import Types, WatermarkStrategy, Duration from
pyflink.datastream import StreamExecutionEnvironment from
pyflink.datastream.connectors.file_system import (     FileSource,
StreamFormat)  env =
StreamExecutionEnvironment.get_execution_environment()
env.set_parallelism(1)  # An UNBOUNDED source: watch the folder, pick up
new files as they appear source = (FileSource
.for_record_stream_format(StreamFormat.text_line_format(),
                              "stream_input")
    .monitor_continuously(Duration.of_seconds(2))

.build())
lines = env.from_source(source,
WatermarkStrategy.no_watermarks(),
                        "dir-
source")  def tokenize(line):
for word in line.split():
        yield (word, 1)  counts = (lines     .flat_map(tokenize,
output_type=Types.TUPLE([Types.STRING(), Types.INT()]))
.key_by(lambda x: x[0])
    .reduce(lambda a, b: (a[0], a[1] + b[1])))  counts.print()
env.execute("streaming_word_count")   # runs forever — Ctrl+C to
stop
```

Task 3 — Start the streaming job (Terminal A)

```
cd C:\Flink flink-
env\Scripts\activate python
wordcount_stream.py
```

The job starts and waits. Unlike SET 3 it does not finish, because the source is unbounded.

Task 4 — Feed the stream by creating files (Terminal B)

In a second terminal, drop text files into the monitored folder. Each new file is a fresh burst of records into the live stream.

```
cd C:\Flink echo flink is fun >
stream_input\a.txt echo flink flink spark
> stream_input\b.txt
```

Sample Output in Terminal A (appears within ~2 seconds of each file):

```
(flink,1)
(is,1)
(fun,1)
(flink,2)
(flink,3)
(spark,1)
```

The count for flink climbs across files (1 -> 2 -> 3) because the running state persists for the life of the job. The very same flat_map / key_by / reduce operators from SET 3 work unchanged on this infinite stream — demonstrating Flink's unified model, in which bounded data is just a special case of unbounded.

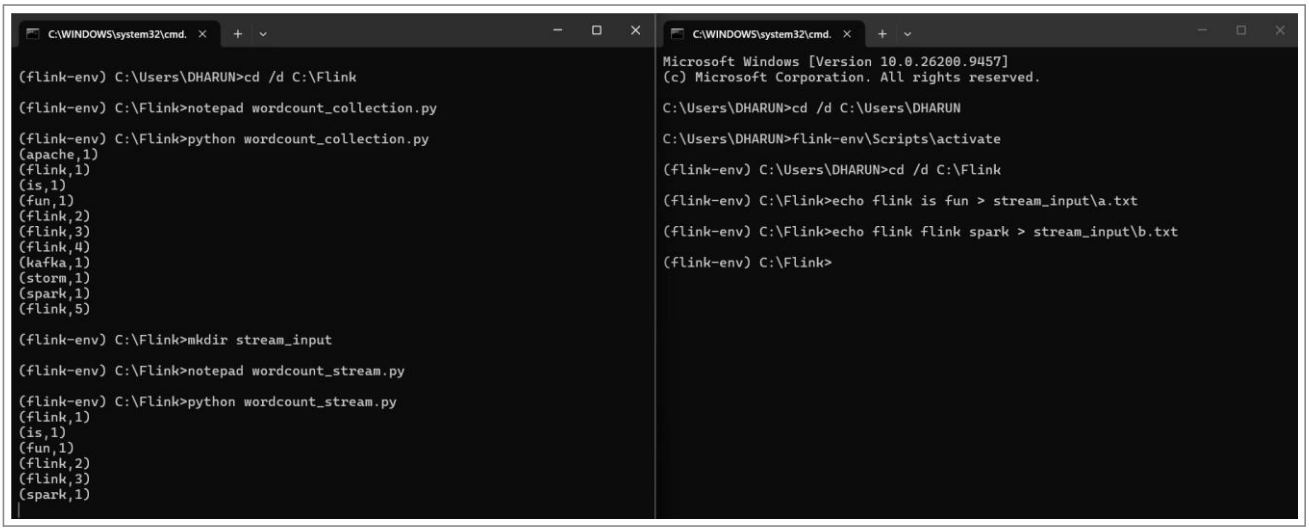


Figure 5: Terminal A (left): streaming job output. Terminal B (right): files fed into stream_input

Result: The identical pipeline processed a continuous, never-ending stream, updating word counts within about two seconds of each new file — a genuine, stateful, real-time stream computation rather than a one-off script.

SET 5: Windowed Word Count (Unique Signature Task)

Goal: *Demonstrate Flink's signature capability — time windows — by counting words in fixed 5-second buckets instead of as a single running total. This is the feature that most clearly sets Flink apart from a plain script and gives streaming its notion of time.*

A window groups the unbounded stream into finite chunks so an aggregate can be computed per chunk. Here a tumbling processing-time window of 5 seconds means every 5 seconds Flink emits the counts for the words seen in that interval, then starts fresh. Only two lines change from SET 4: an import and a .window(...) call before reduce.

Task 1 — Create wordcount_windowed.py

```
from pyflink.common import Types, WatermarkStrategy, Duration,
Time from pyflink.datastream import StreamExecutionEnvironment
from pyflink.datastream.connectors.file_system import (
FileSource, StreamFormat) from pyflink.datastream.window import
TumblingProcessingTimeWindows  env =
StreamExecutionEnvironment.get_execution_environment()
env.set_parallelism(1)  source = (FileSource
.for_record_stream_format(StreamFormat.text_line_format(),
                              "stream_input")
    .monitor_continuously(Duration.of_seconds(2))

.build())
lines = env.from_source(source,
WatermarkStrategy.no_watermarks(),
                        "dir-
source")  def tokenize(line):
for word in line.split():
        yield (word, 1)  counts = (lines     .flat_map(tokenize,
output_type=Types.TUPLE([Types.STRING(), Types.INT()]))
.key_by(lambda x: x[0])
    .window(TumblingProcessingTimeWindows.of(Time.seconds(5)))    # NEW
    .reduce(lambda a, b: (a[0], a[1] +
b[1])))  counts.print()
env.execute("windowed_word_count")
```

Task 2 — Run it and feed files in different 5-second windows

Start the job in Terminal A, then in Terminal B drop one file, wait several seconds so the window closes, then drop another. Each file lands in a different window.

```
REM Terminal A: python wordcount_windowed.py  REM
Terminal B (wait ~6 seconds between the two commands):
echo flink flink spark > stream_input\a.txt echo flink
storm flink flink > stream_input\b.txt
```

Sample Output (one group per 5-second window):

```
(flink,2)
(spark,1)
(flink,3)
(storm,1)
```

The contrast with SET 4 is the whole point. In SET 4 the count was a single running total that only grew (flink reached 3, then would keep climbing). Here each window is counted independently: the first window reports flink twice, and the second window reports flink three times — the count resets per window rather than accumulating forever. This per-interval aggregation over time is exactly what windowing provides, and it is the foundation of real streaming analytics (counts per minute, averages per hour, and so on).

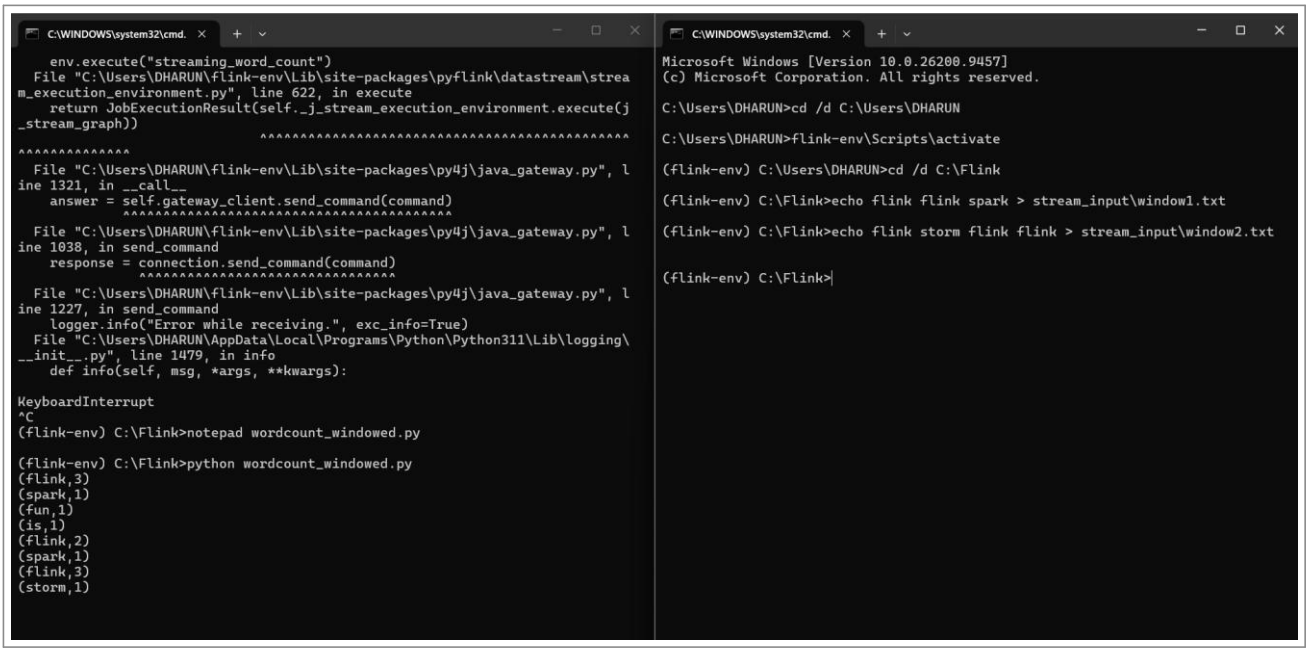


Figure 6: Terminal A (left): windowed job output. Terminal B (right): files fed in different 5-second windows

Result: The same stream was aggregated into independent 5-second windows, each emitting its own word counts — demonstrating Flink's time-based windowing, the capability that distinguishes it as a full stream-processing engine rather than a simple continuous script.

Overall Conclusion

Apache Flink was installed and run natively on Windows through its Python API, PyFlink, inside an isolated Python 3.11 virtual environment, with no separate cluster required because PyFlink launches an embedded mini-cluster on execution. Using the DataStream API's core operators — flat_map, key_by and reduce — a word-frequency application was built and run in three progressively richer forms: over a bounded in-memory collection, over an unbounded continuously-monitored directory that produced live running counts, and finally over 5-second tumbling windows that produced independent per-interval counts. The identical operators worked across bounded and unbounded sources, illustrating Flink's unifying principle that a batch is merely a bounded stream. Together with Experiment 1 (Kafka messaging) and Experiment 2 (Storm topologies), this experiment completes the picture of a modern streaming stack — reliable message delivery feeding continuous, stateful, windowed computation — and directly supports CO1 and CO2 of the course.